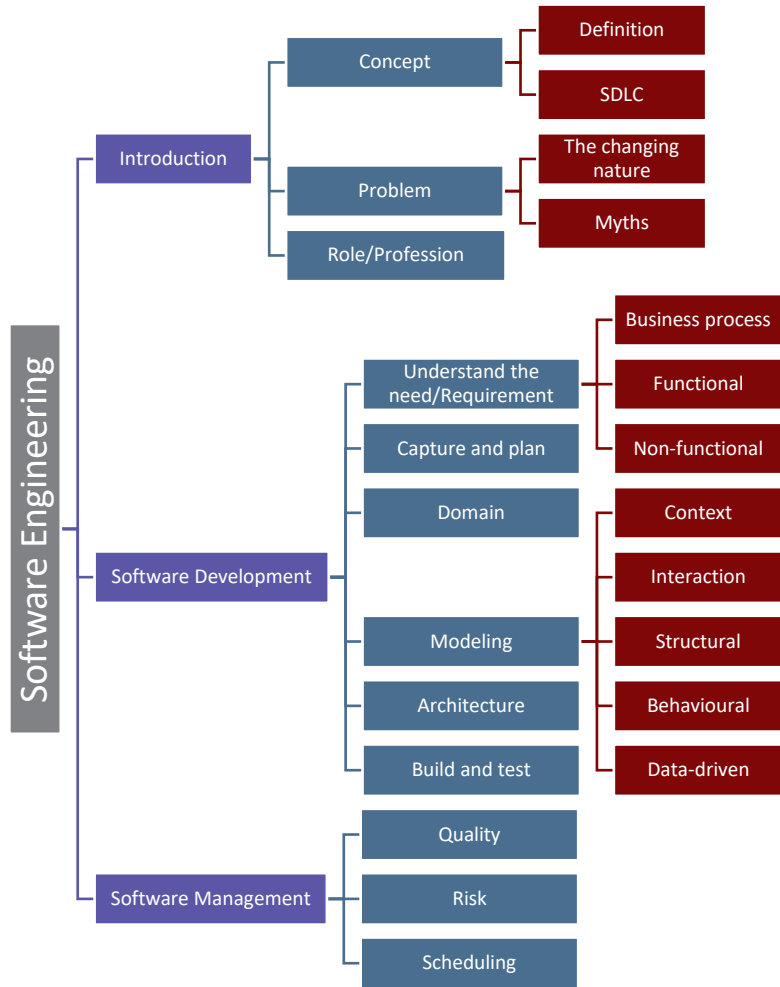




SW Development Process: Life Cycles & Methodologies

IBDA3011 Map



Session 5 Outcome

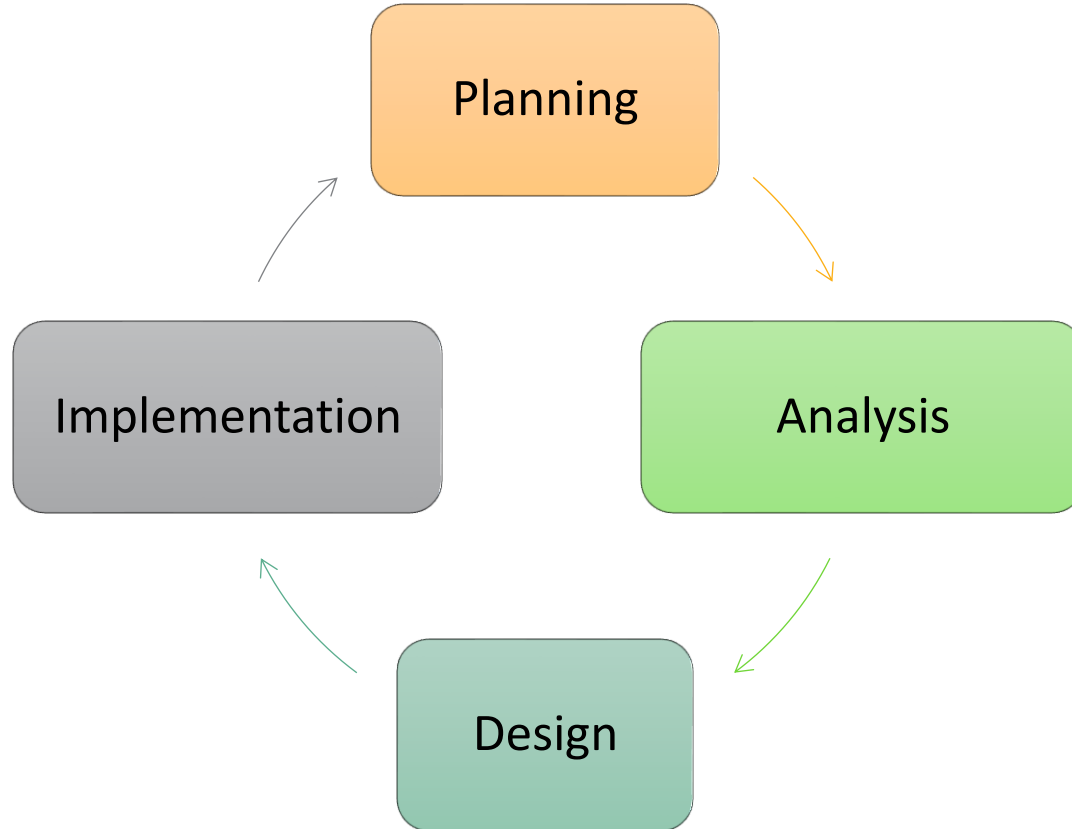
Students are able to explain:

- Software Development Life Cycle (SDLC)
- Software Development Methodologies

Materi

- Software Development Life Cycle (SDLC)
- Software Development Methodologies

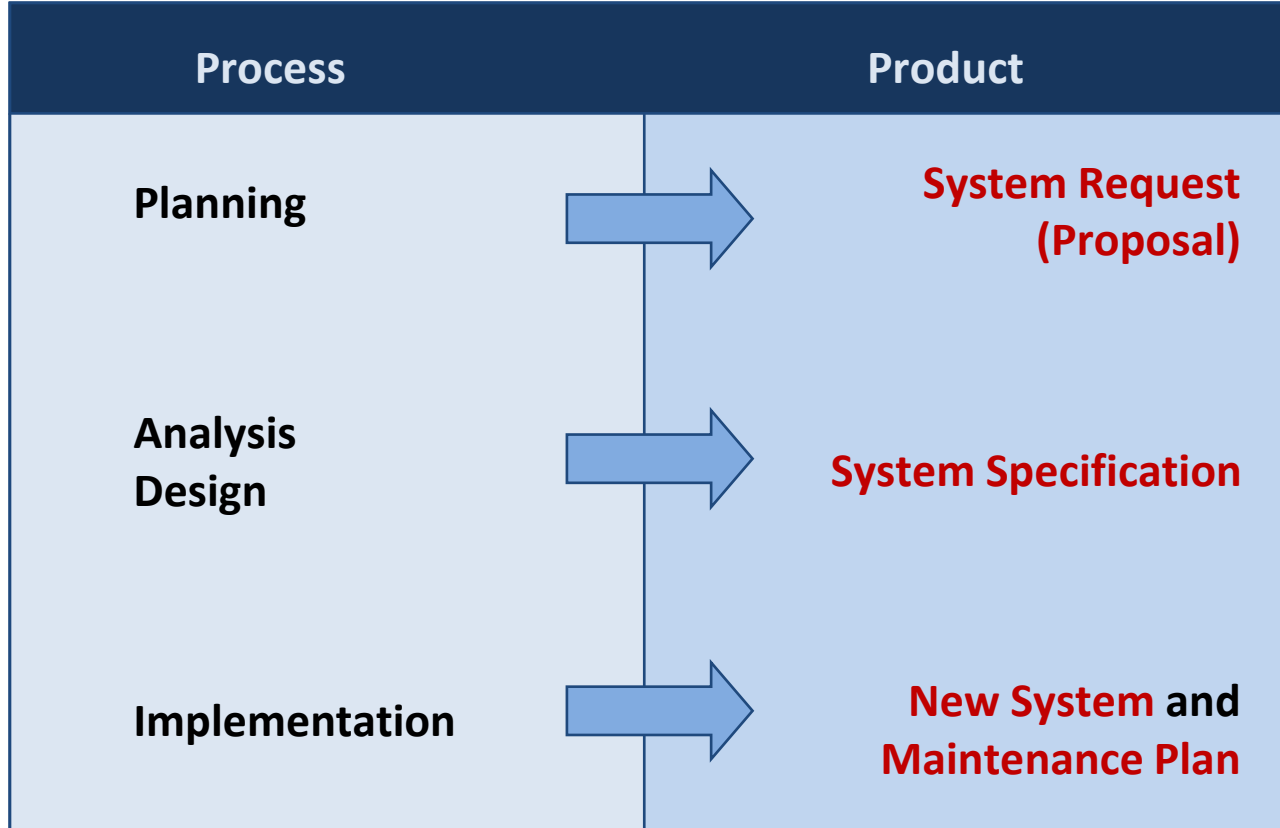
Software Development Life Cycle (SDLC)



Project Phases

- 1. Planning:** Why build the system?
 - Identifying Business Value, Feasibility Analysis (System Request/Proposal)
- 2. Analysis:** Who, what, when, where will the system be?
 - User Requirement Gathering (System Specification)
- 3. Design:** How will the system work?
 - Architecture Design, Interface Design, Data Design, Program Design (System Specification)
- 4. Implementation:** System delivery
 - System Construction, Testing, Installation, Maintenance (New System)

Processes and Deliverables



Iterative development is an approach to system development in which the system is "grown" piece by piece through multiple iterations

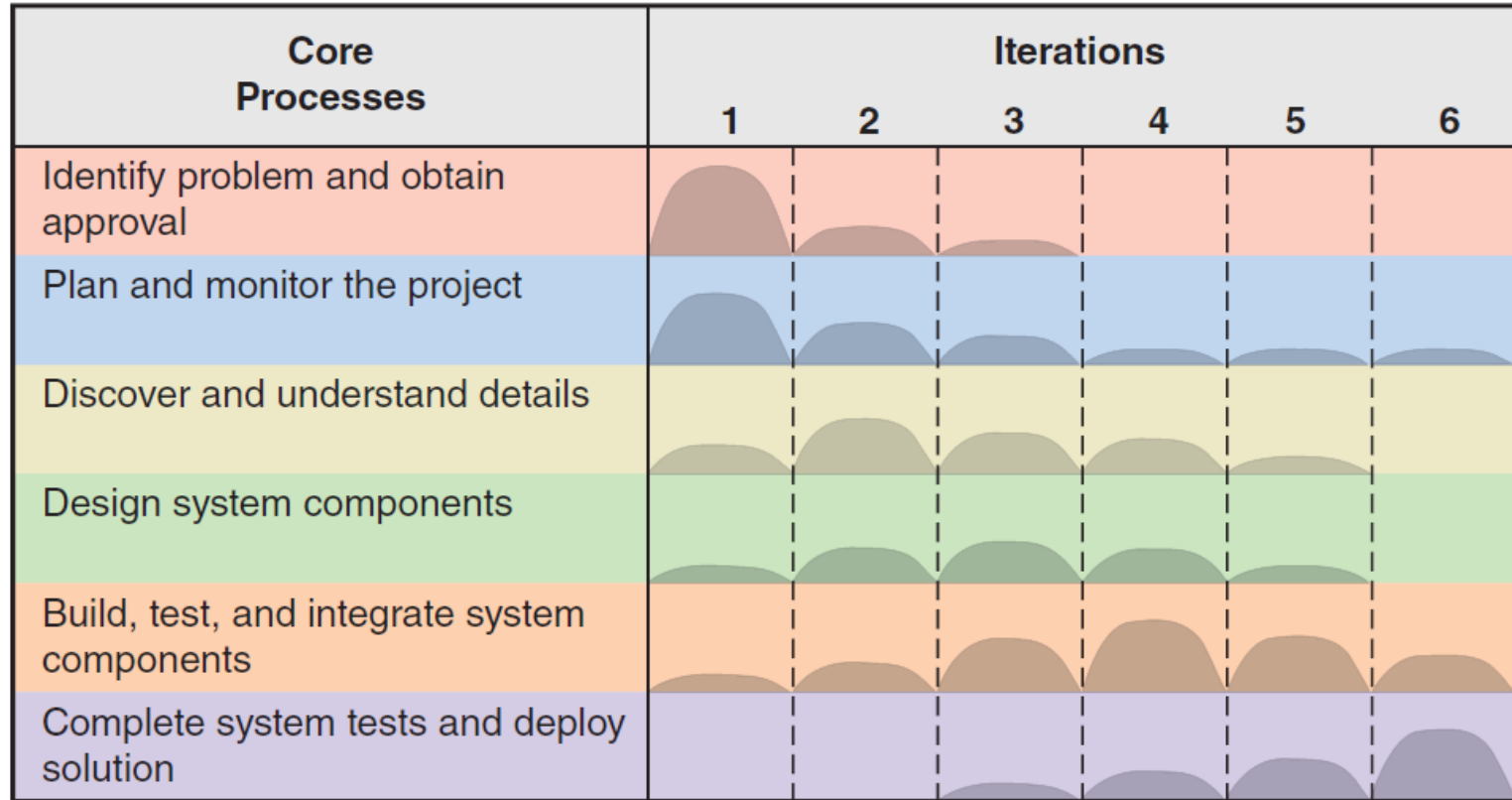
The six core

6 Core

six core processes are required in the development of any new application:

1. Identify the problem or need and obtain approval to proceed.
2. Plan and monitor the project—what to do, how to do it, and who does it.
3. Discover and understand the details of the problem or the need.
4. Design the system components that solve the problem or satisfy the need.
5. Build, test, and integrate system components.
6. Complete system tests and then deploy the solution.

The six core processes with iterations for a typical project



Software Development Methodologies

What Is a Methodology?

- A formalized **approach to implementing** the SDLC (series of steps and deliverables)
- Writing code **without a well-thought-out** system request may work for small programs, but **rarely works for large ones**

Major Methodologies

1. Structured Design

- Waterfall Method
- Parallel Development

2. Rapid Application Development (RAD)

- Phased Development
- Prototyping
- Throw-away Prototyping

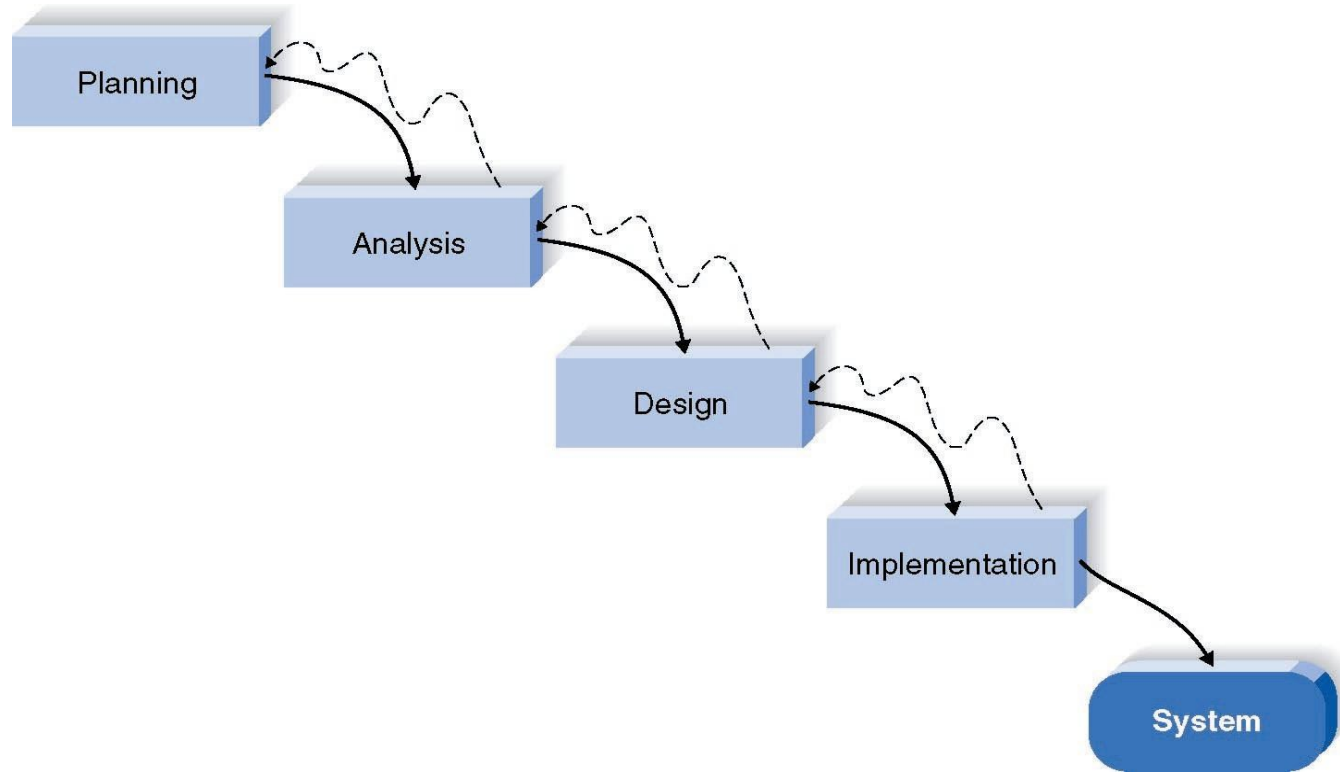
3. Agile Development

- Extreme Programming (XP)
- Scrum

Structured Design Methodology

- Projects **move methodically** from one to the next step
- Generally, **a step is finished before the next one begins**

Waterfall Method



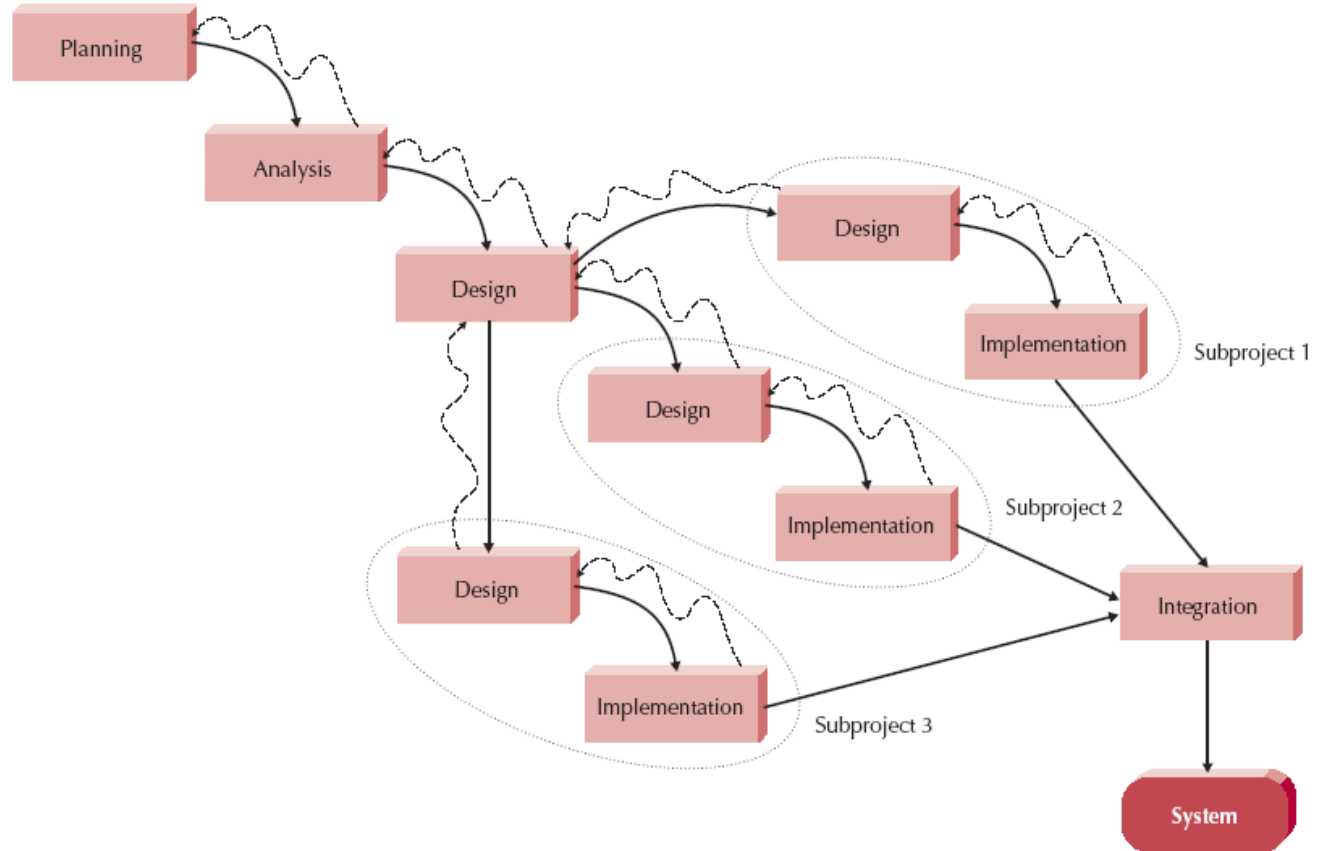
Pros - Cons of the Waterfall Method

Pros	Cons
Identifies systems requirements long before programming Begins, it minimizes change to the requirements as the project proceed (mature)	Design must be specified on paper before programming begins
	Long time between system proposal and delivery of new system
	Rework is very hard

Parallel Development

- Addresses problem of time gap between proposal and delivery
- General process:
 1. Breaks project into parallel subproject
 2. Integrates them at the end

Parallel Development



Rapid Application Development

1. Phased development

- A series of versions

2. Prototyping

- System prototyping

3. Throw-away prototyping

- Design prototyping

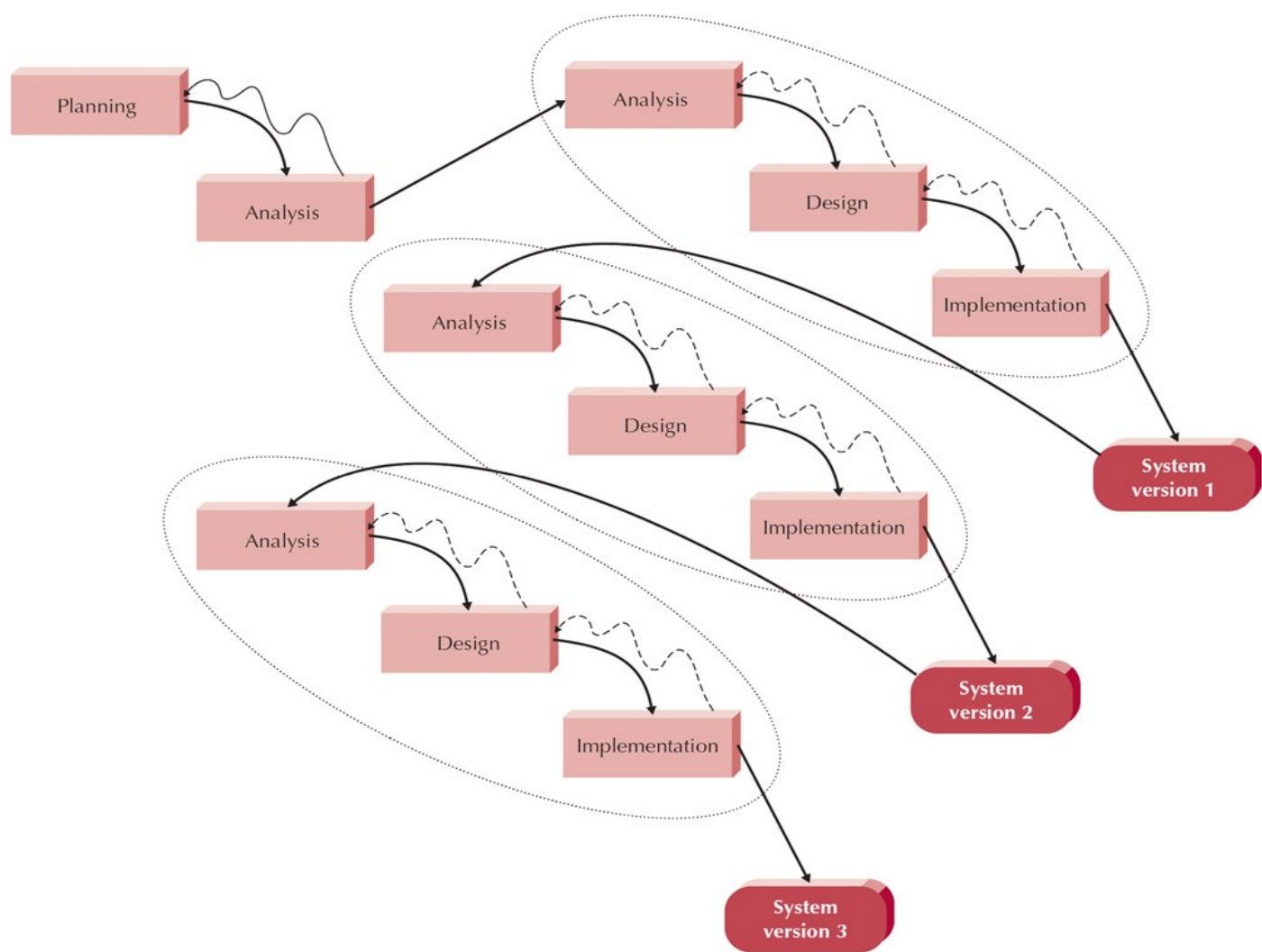
Rapid Application Development

Critical elements to speed up the SDLC:

1. CASE tools
2. Visual programming languages
3. Code generators

RAD: Phased Development

- Break overall system into a series of versions
- Each version has Analysis, Design, and Implementation
- Output from one version is the input to the next
- Incorporate ideas, issues, lessons learned in one version into the next version



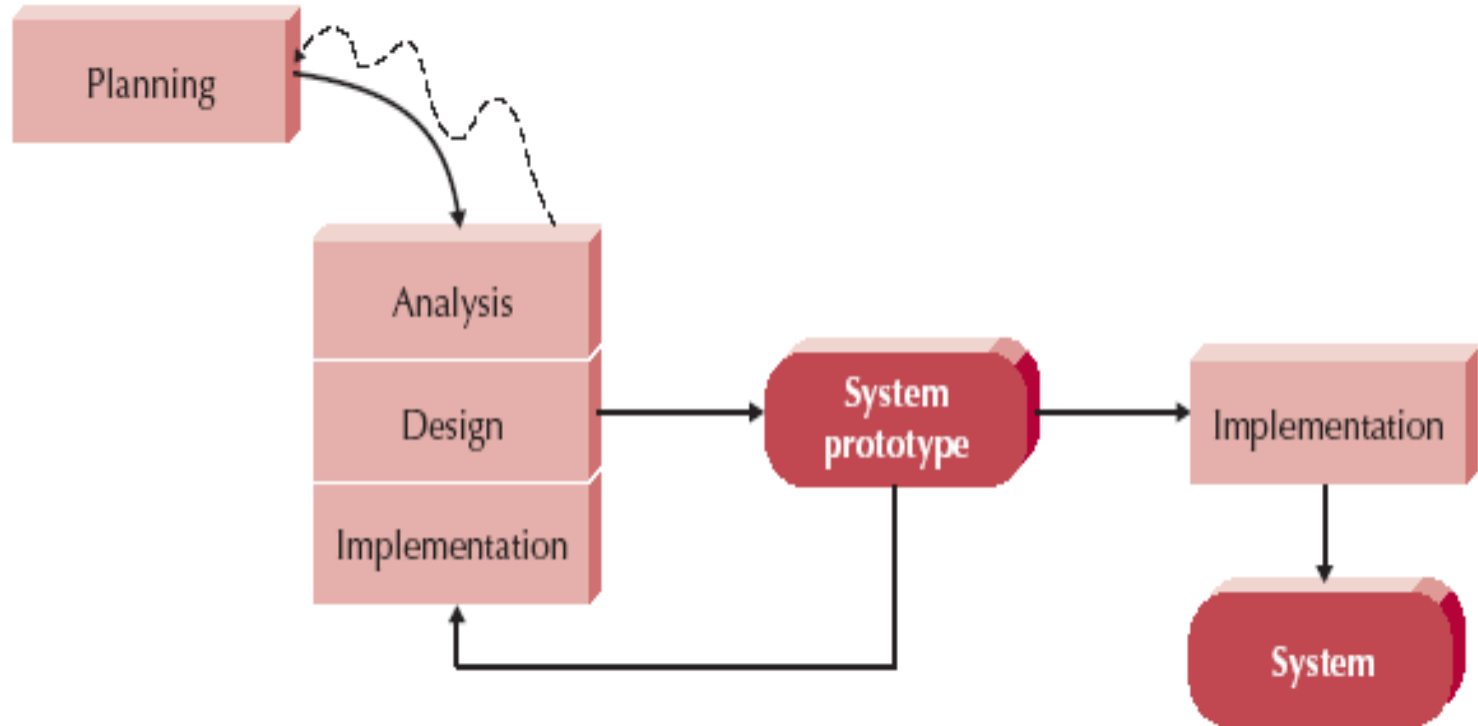
RAD: Phased Development

Pros	Cons
Gets useful system to users quickly Most important functions tested most	Initial system is intentionally incomplete System requirements expand as users see versions

RAD: Prototyping

- Analysis, Design, Implementation are performed concurrently
- Start with a "quick-and-dirty" prototype
 - Provides minimal functionality
- Repeat process, refining the prototype each time
- Stop when prototype is a working system

RAD: Prototyping



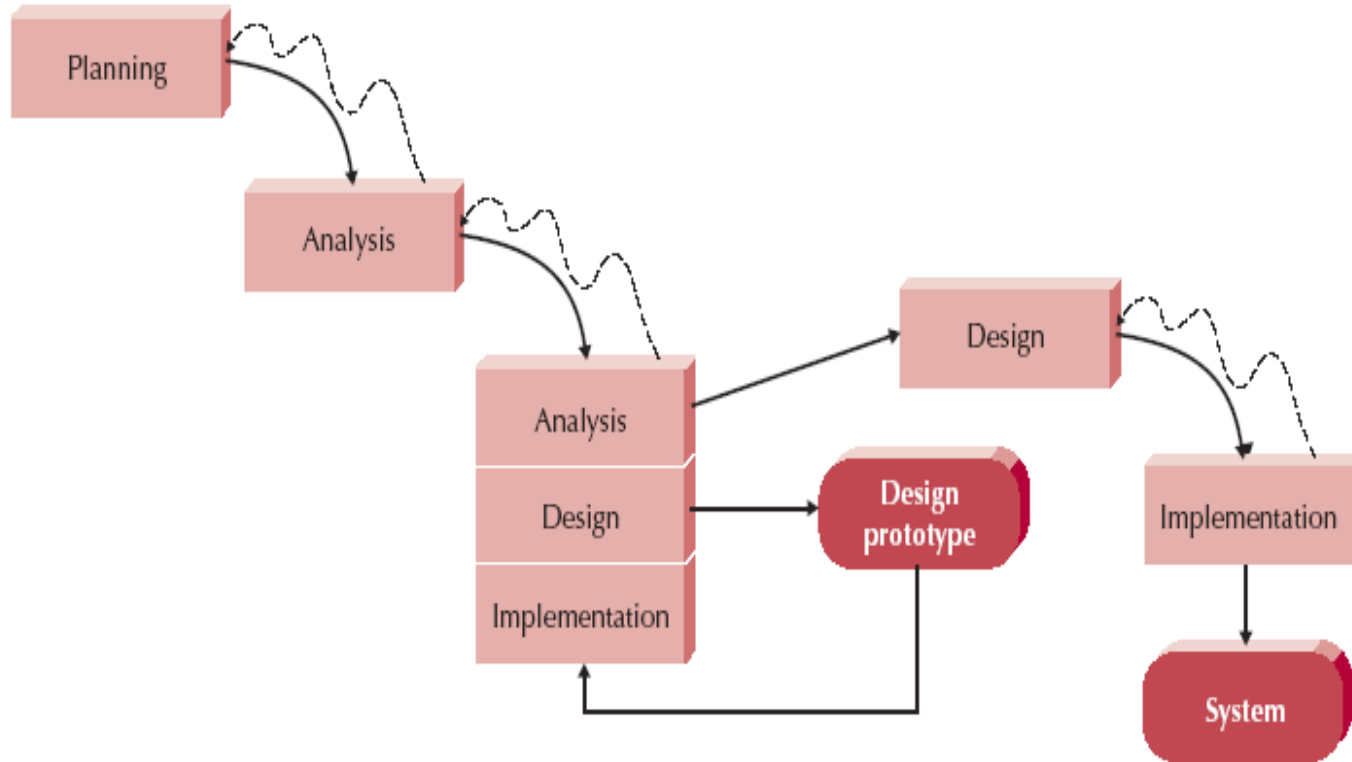
RAD: Prototyping

Pros	Cons
Gets working system to users quickly	Fast paced. Hard to conduct careful, methodical analysis
Reassures users that the project is progressing	Initial design decisions have long term staying power
Quickly refines true requirements	Problems may come to light late in design, requiring re-design

RAD: Throw-Away Prototyping

- Use prototypes only to understand requirements
 - Example: use html to show UI
- Prototype is **not** a working design
- Once requirements are understood, the prototypes are thrown away
- The system is then built using SDLC

RAD: Throw-Away Prototyping



Agile Development

- Just a **few rules** that are easy to learn and follow
- Streamline the SDLC
 - **Eliminate** much of the **modeling and documentation**
 - Emphasize **simple, iterative** application development
- Examples include:
 - **Extreme Programming** (XP)
 - **Scrum**
 - Dynamic Systems Development Model (DSDM)

Extreme Programming (XP)

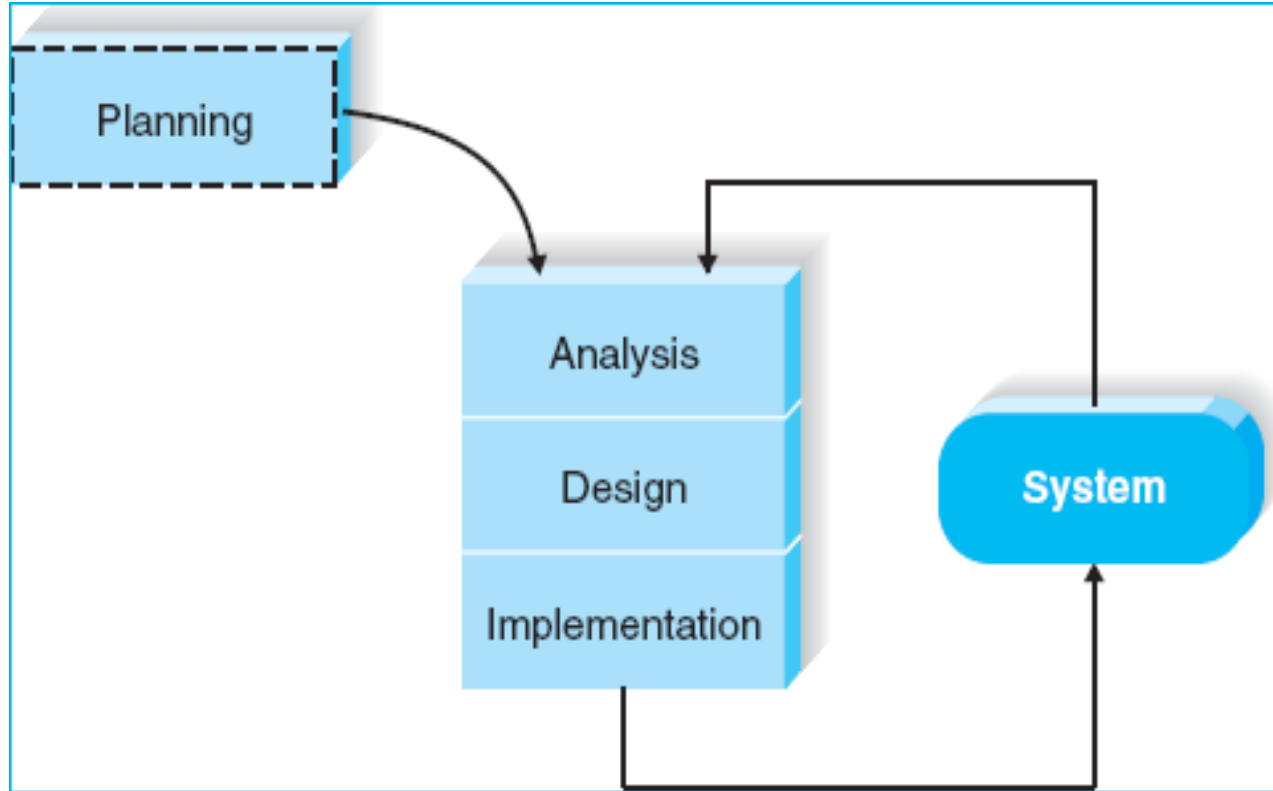
“Core Values” of XP

1. Communication – All to All
2. Simplicity – KISS, refactoring
3. Feedback – Embrace Change
4. Courage – Quality First, test and efficient coding

Extreme Programming (XP)

1. User Stories about system do
2. Code small program
3. User Feedback
4. Repeat
5. Standards are important
 - Naming conventions
 - Coding practices

Extreme Programming



Selecting the Appropriate Methodology

1. Clarity of User Requirements
2. Familiarity with Technology
3. System Complexity
4. System Reliability
5. Short Time Schedules
6. Schedule Visibility

Selecting the Right Methodology

Ability to Develop Systems	Structured Methodologies			RAD Methodologies		Agile Methodologies
	Waterfall	Parallel	Phased	Prototyping	Throwaway Prototyping	XP
with Unclear User Requirements	Poor	Poor	Good	Excellent	Excellent	Excellent
with Unfamiliar Technology	Poor	Poor	Good	Poor	Excellent	Poor
that are Complex	Good	Good	Good	Poor	Poor	Poor
that are Reliable	Good	Good	Good	Poor	Poor	Good
with a Short Time Schedule	Poor	Good	Excellent	Excellent	Good	Excellent
with Schedule Visibility	Poor	Poor	Excellent	Excellent	Good	Good

Selecting Methodology

Suppose you are an analyst for the Roanoke Software Consulting Company (RSCC), a large consulting firm with offices around the world. The company wants to build a new knowledge management system that can identify and track the expertise of individual consultants anywhere in the world based on their education and the various consulting projects on which they have worked. Assume that this is a new idea that never done before been attempted in RSCC or elsewhere. RSCC has an international network, but the offices in each country may use somewhat different hardware and software. RSCC management wants the system up and running within a year.

5 Problems in the Software Development

1. **Poor Requirements** - if requirements are unclear, incomplete, too general, and not testable, there may be problems.
2. **Unrealistic Schedule** - if too much work is crammed in too little time, problems are inevitable.
3. **Inadequate Testing** - no one will know whether or not the software is any good until customers complain or systems crash.
4. **Featuritis** - requests to add on new features after development goals are agreed on.
5. **Miscommunication** - if developers don't know what's needed or customer's have erroneous expectations, problems can be expected.

5 Solutions in the Software Development

1. **Solid Requirements** - clear, complete, detailed, cohesive, attainable, testable requirements that are agreed to by all players. In 'agile'-type environments, continuous close coordination with customers/end-users is necessary to ensure that changing/emerging requirements are understood.

5 Solutions in the Software Development

2. **Realistic Schedules** - allow adequate time for planning, design, testing, bug fixing, re-testing, changes, and documentation; personnel should be able to complete the project without burning out.

5 Solutions in the Software Development

3. **Adequate Testing** - start testing early on, re-test after fixes or changes, plan for adequate time for testing and bug-fixing. 'Early' testing could include static code analysis/testing, test-first development, unit testing by developers, built-in testing and diagnostic capabilities, automated post-build testing, etc.

5 Solutions in the Software Development

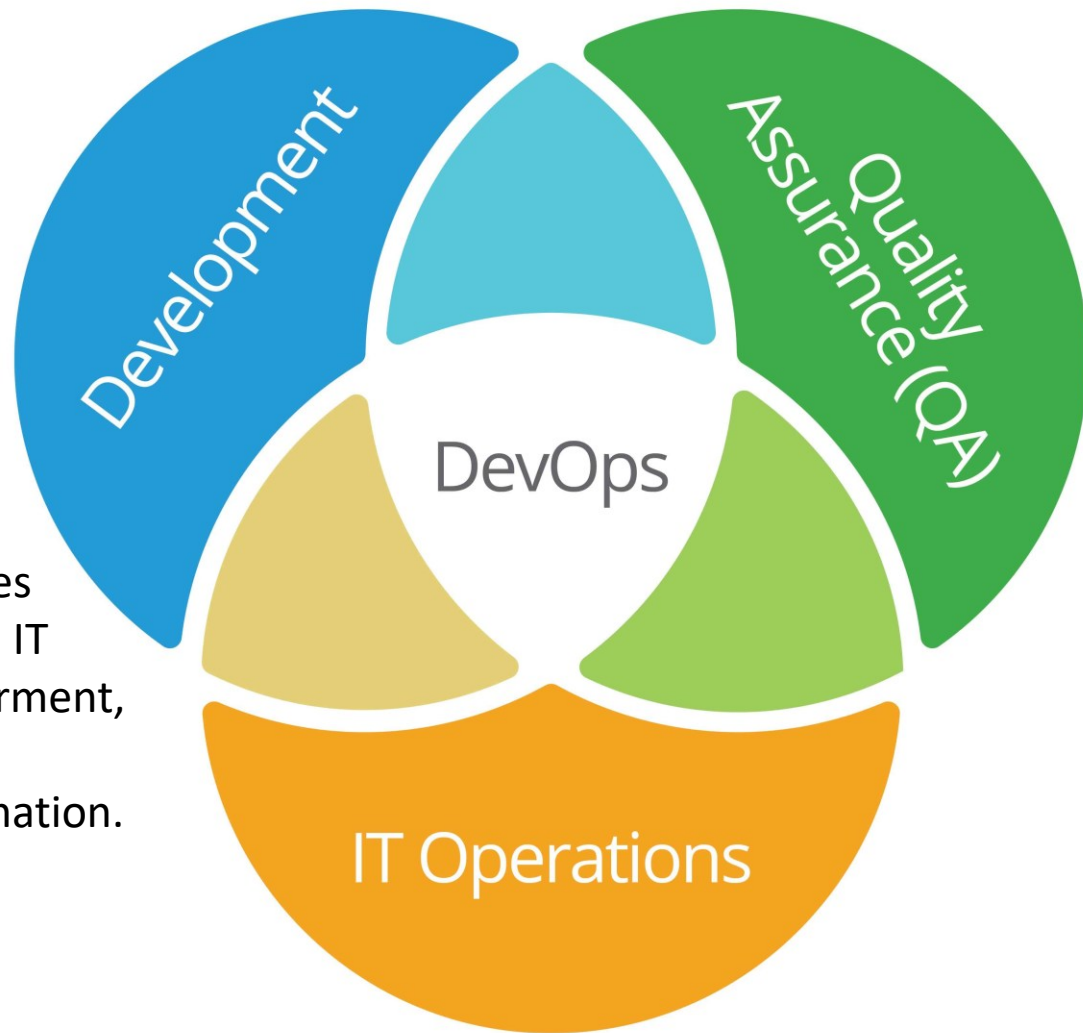
4. **Stick to Initial Requirements where Feasible** - be prepared to defend against excessive changes and additions once development has begun, and be prepared to explain consequences. If changes are necessary, they should be adequately reflected in related schedule changes. If possible, work closely with customers/end-users to manage expectations. In 'agile'-type environments, initial requirements may be expected to change significantly, requiring that true agile processes be in place and followed.

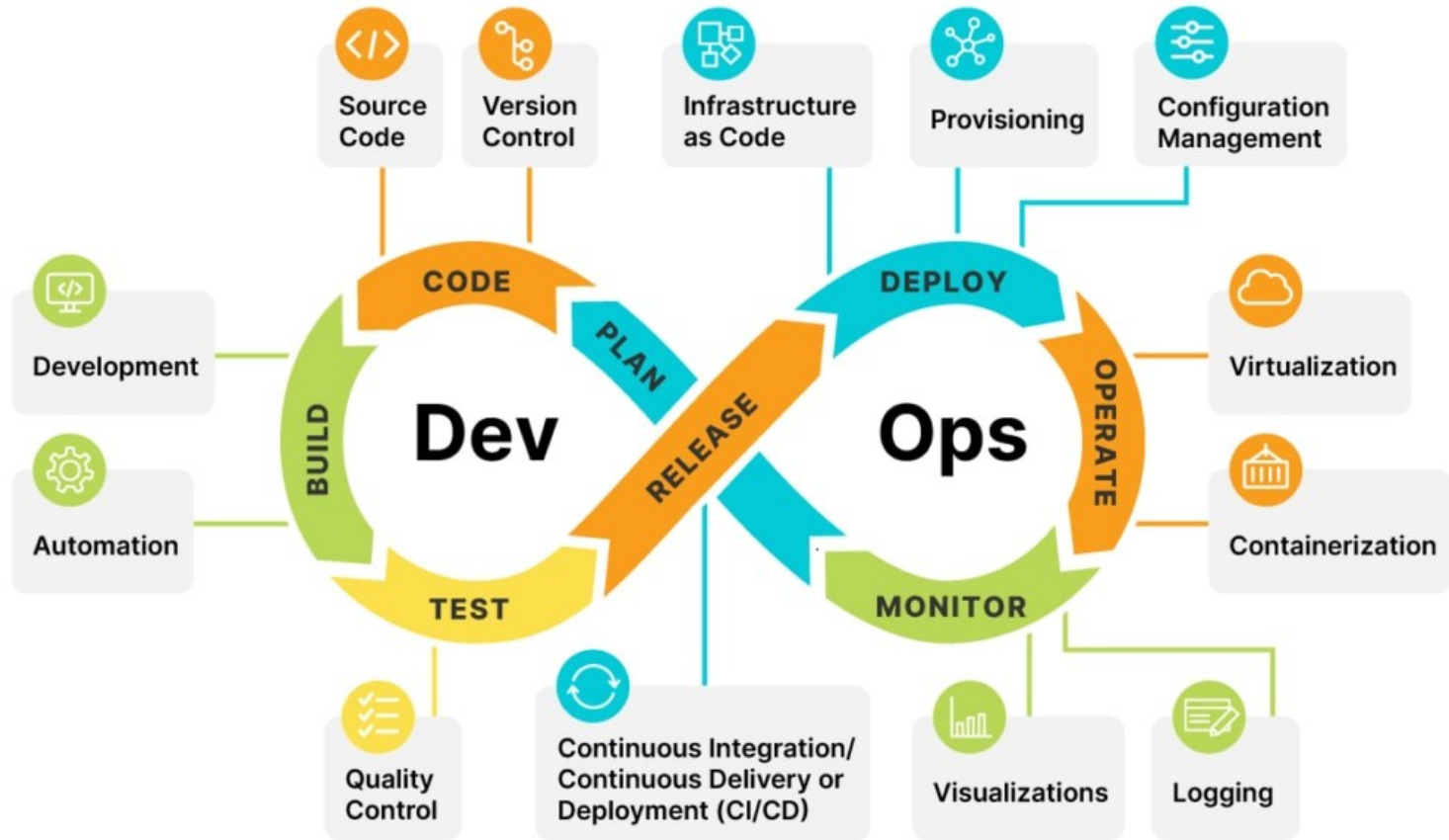
5 Solutions in the Software Development

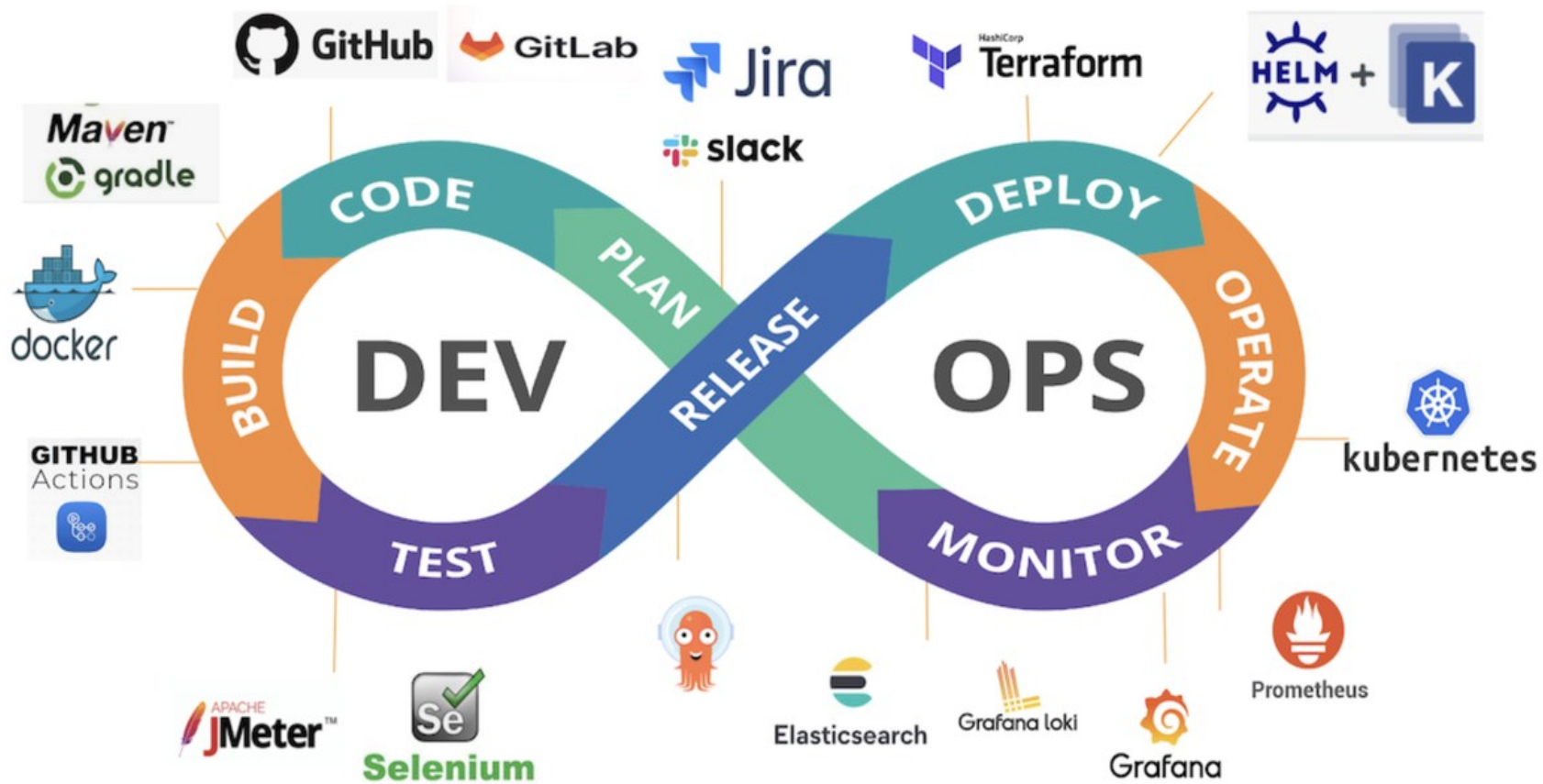
5. **Communication** - require walkthroughs and inspections when appropriate; make extensive use of group communication tools - groupware, wiki's, bug-tracking tools and change management tools, intranet capabilities, etc.; ensure that information/documentation is available and up-to-date - preferably electronic, not paper; promote teamwork and cooperation; use prototypes and/or continuous communication with end-users if possible to clarify expectations.

DevOps

DevOps is a **set of practices, tools,** and a cultural **philosophy** that automate and integrate the processes between software development and IT teams. It emphasizes team empowerment, cross-team communication and collaboration, and technology automation.







What are the benefits of DevOps?

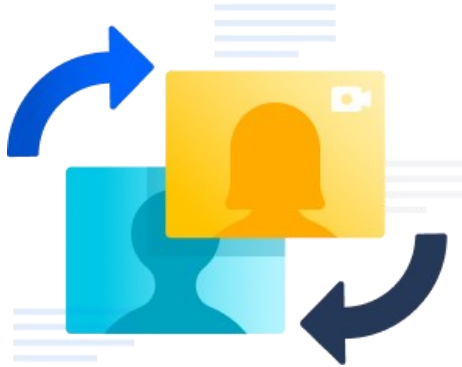


Speed

Teams that practice DevOps release deliverables more frequently, with higher quality and stability.

In fact, the DORA 2019 State of DevOps report found that elite teams deploy 208 times more frequently and 106 times faster than low-performing teams. Continuous delivery allows teams to build, test, and deliver software with automated tools.

What are the benefits of DevOps?



Improved collaboration

The foundation of DevOps is a culture of collaboration between developers and operations teams, who share responsibilities and combine work. This makes teams more efficient and saves time related to work handoffs and creating code that is designed for the environment where it runs.

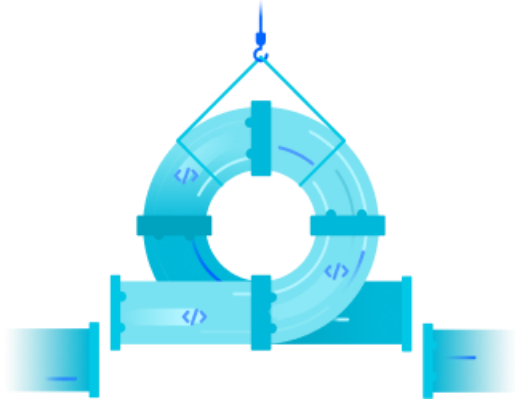
What are the benefits of DevOps?



Rapid deployment

By increasing the frequency and velocity of releases, DevOps teams improve products rapidly. A competitive advantage can be gained by quickly releasing new features and repairing bugs.

What are the benefits of DevOps?



Quality and reliability

Practices like continuous integration and continuous delivery ensure changes are functional and safe, which improves the quality of a software product. Monitoring helps teams keep informed of performance in real-time.

What are the benefits of DevOps?



Security

By integrating security into a continuous integration, continuous delivery, and continuous deployment pipeline, DevSecOps is an active, integrated part of the development process. Security is built into the product by integrating active security audits and security testing into agile development and DevOps workflows.

DevOps isn't any single person's job.
It's everyone's job.

Robert Krohn
HEAD OF ENGINEERING, DEVOPS AT ATlassian



Summary

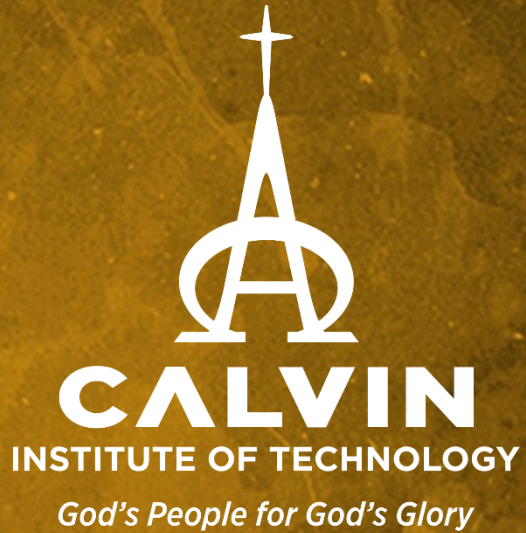
- **Software Engineering Process:** Covers the Software Development Life Cycle (SDLC), including planning, analysis, design, and implementation phases. It also discusses the changing nature of software engineering and common myths.
- **Software Development:** Focuses on understanding requirements, both functional and non-functional, and various modeling techniques such as context, interaction, structural, and behavioral models.
- **Software Management:** Discusses quality, risk, and scheduling in software projects. It also includes the importance of DevOps practices for integrating development and IT operations.
- **Pros and Cons:** Lists the advantages and disadvantages of different software development methodologies, emphasizing the balance between quick delivery and thorough analysis.

Key Insight

- **Software Engineering Process:** Covers the **Software Development Life Cycle (SDLC)**, including planning, analysis, design, and implementation phases.
- **Software Development:** Emphasizes understanding requirements, both functional and non-functional, and includes various modeling techniques like context, interaction, structural, and behavioral.
- **Software Management:** Focuses on quality, risk, and scheduling, highlighting the importance of planning and maintaining new systems.
- **DevOps:** Described as a set of practices and tools that integrate software development and IT operations, emphasizing **team empowerment, communication, collaboration, and automation.**

Exercises

- What are the key phases of the Software Development Life Cycle (SDLC)?
- How does DevOps integrate with traditional software engineering processes?
- What are the pros and cons of different software development methodologies mentioned?
- What roles and responsibilities are highlighted for software engineers in this document?



Thank You